



Applied Software Project Report

By

Lijin Shaji

**A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements
for the degree of Master of Science in Computer Science**

June, 2025



Scaler Mentee Email ID : lijinshajii@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 11/06/2025

© The project report of Lijin Shaji is approved, and it is acceptable in quality and form for publication electronically

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

.....

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 06/04/2024 to 20/11/2024, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Lijin Shaji



Signature of the Candidate

Date: 11/06/2025

ACKNOWLEDGMENT

I want to take a moment to express my deepest gratitude to everyone who helped me reach this incredible milestone. First and foremost, to my family, your unwavering support, encouragement, and understanding have been the bedrock of my journey. This achievement wouldn't have been possible without you.

A special thank you goes out to my incredible instructors at Scaler, particularly Naman Bhalla and Soumodeep Bhowmick. Your expertise, guidance, and passion for teaching have not only enriched my knowledge but also truly inspired me throughout this program.

And to my batchmates, thank you for the camaraderie, the shared struggles, and the collective spirit that made this program a truly memorable experience. Your support and collaboration were invaluable.

Finally, to everyone else who offered a helping hand, a word of motivation, or a spark of inspiration along the way, thank you. Your contributions, big or small, have made a significant difference. I'm truly grateful for all of you.

Table of Contents

Contents

List of Tables	6
List of Figures	7
Applied Software Project.....	8
Abstract.....	8
Project Description	9
Project Objectives.....	9
Requirement Gathering	11
Feature Set	13
Class Diagrams.....	15
User Class Diagram.....	16
Controller Class Diagram.....	18
Service Class Diagram	21
Database Schema Design	28
Feature Development Process	33
Deployment Flow	34
Technologies Used	40
Conclusion	42
References.....	43

List of Tables

(To be written sequentially as they appear in the text)

Table No.	Title	Page No.
1	Feature table	14

List of Figures

(List of Images, Graphs, Charts sequentially as they appear in the text)

Figure No.	Title	Page No.
1	Project Diagrams	10
2	Class Diagrams	12, 16, 18, 21
3	Database schema	28
4	Creation of JAR File	34, 35
5	Elastic Beanstalk images	37
6	RDS Images	38

Applied Software Project

Abstract

This project is a backend eCommerce application developed using Java Spring Boot, designed to manage products, carts, orders, and user authentication in a secure and structured manner.

The system supports full product management, allowing administrators to add and view products categorized under different sections. Each product includes details like title, description, price, images, and rating.

Users can interact with the cart module to add, update, or remove products, and proceed to checkout. Once a purchase is initiated, the order management system captures details including cart content, user information, total price, and payment status.

For security, the application implements JWT-based authentication, allowing users to securely register, log in, and access authorized features. Token-based access control ensures only authenticated users can perform sensitive operations.

The backend leverages Spring Boot, Spring Security, JPA/Hibernate, and MySQL, following RESTful principles and clean architectural practices. The project is modular, scalable, and easily extendable for integration with frontend frameworks or mobile applications.

Project Description

This project is a backend system for an eCommerce platform developed using **Java Spring Boot**. It is designed to handle and manage essential eCommerce functions such as product management, user authentication, cart handling, and order processing in a structured and secure way.

The system provides RESTful APIs for frontend or mobile clients and ensures high performance, modularity, and security through the use of Spring Boot, Spring Security, JPA/Hibernate, and MySQL.

Project Objectives

- **Product Management:** Enable admin users to create, view, and categorize products with details like title, price, description, rating, and images.
- **Cart Management:** Allow users to add, update, and remove products from their cart.
- **Order Management:** Facilitate order placement by capturing cart data, user information, total cost, and payment status.
- **Authentication & Authorization:** Implement JWT-based login/registration to protect user data and secure APIs.
- **RESTful Architecture:** Develop a clean, modular API layer to be easily consumed by frontend or mobile applications.
- **Scalability & Maintainability:** Design with extensibility in mind to support future features like payment gateways, reviews, and notifications.

eCommerce continues to dominate the digital marketplace. This backend system demonstrates the core principles of modern web application development:

- Secure handling of user data and transactions.
- Clean and scalable microservice-ready backend architecture.
- Practical implementation of real-world backend features.
- A strong base for learners and professionals interested in full-stack development.

This project can serve as a foundational blueprint for anyone looking to build or understand scalable eCommerce systems using industry-standard technologies.

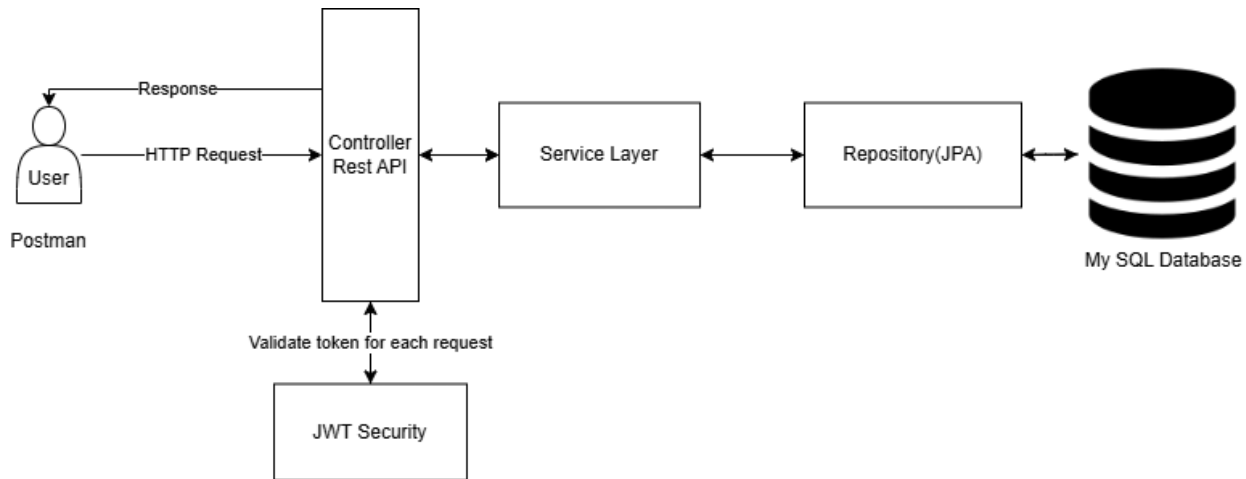


Figure 1.1: System Architecture of ecommerce

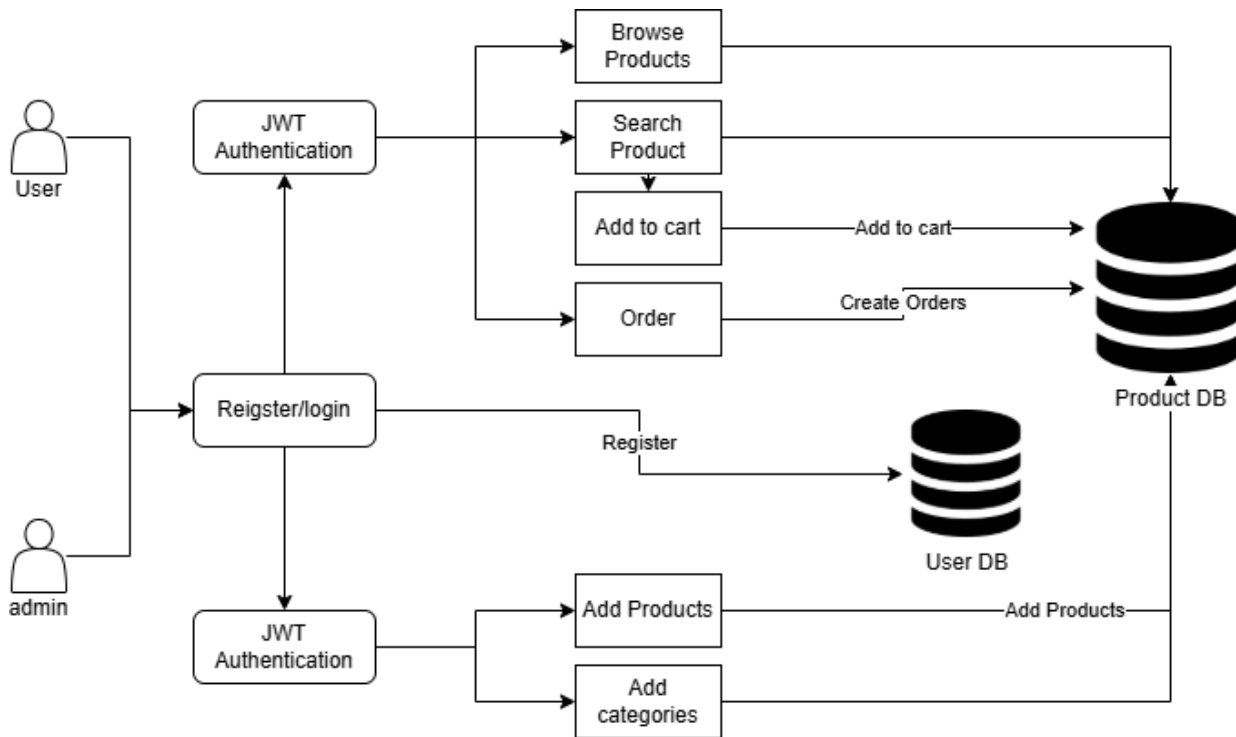


Figure 1.2: user flow chart

Requirement Gathering

Functional Requirements:

- Users can register using their email and password.
- Users can log in and receive a JWT token for secure authentication.
- JWT tokens are required to access protected backend APIs.
- Admin users can add products.
- Admin users can create and manage product categories.
- Users can browse available products and filter them by category.
- Users can search for products product titles and categories.
- Users can add products to their cart for later purchase.
- Users can update item quantities and delete cart.
- Users can place an order using the products in their cart.
- The system records complete order details including items, total price, and user information.

Non - functional Requirements:

- All protected APIs must be secured using JWT-based authentication.
- User passwords should be securely hashed before being stored in the database.
- Responses for common operations like product browsing or searching should be returned within 500 milliseconds.
- The backend must follow a modular and layered architecture (Controller, Service, Repository).
- The application should be easily extendable to integrate with frontend frameworks or mobile applications.
- The system uses MySQL as the database and JPA (Hibernate) for object-relational mapping and persistence.

This eCommerce backend system primarily interacts with two types of users:

- **Customer (User):**

A regular user who registers, browses products, manages their cart, places orders, and tracks past purchases.

- **Administrator (Admin):**

A privileged user who manages product listings, categories, and oversees customer orders through admin-level access and controls.

Use Cases

The key use cases based on user roles are as follows:

For Customers:

- **Register & Login:**

Users can create an account and log in to receive a secure JWT token.

- **Browse Products:**

Users can explore available products by category or through search functionality.

- **Manage Cart:**

Users can add, update, or remove items from their shopping cart.

- **Place Orders:**
Users can checkout with selected items and submit an order.
- **View Order History:**
Users can view previous orders, including item details and payment status.

For Administrators:

- **Admin Authentication:**
Admins can securely log in to access restricted admin features.
- **Product Management:**
Admins can add new products, update existing ones, or delete items.
- **Category Management:**
Admins can create and manage categories to organize products.
- **View Orders:**
Admins can monitor and review orders placed by users.

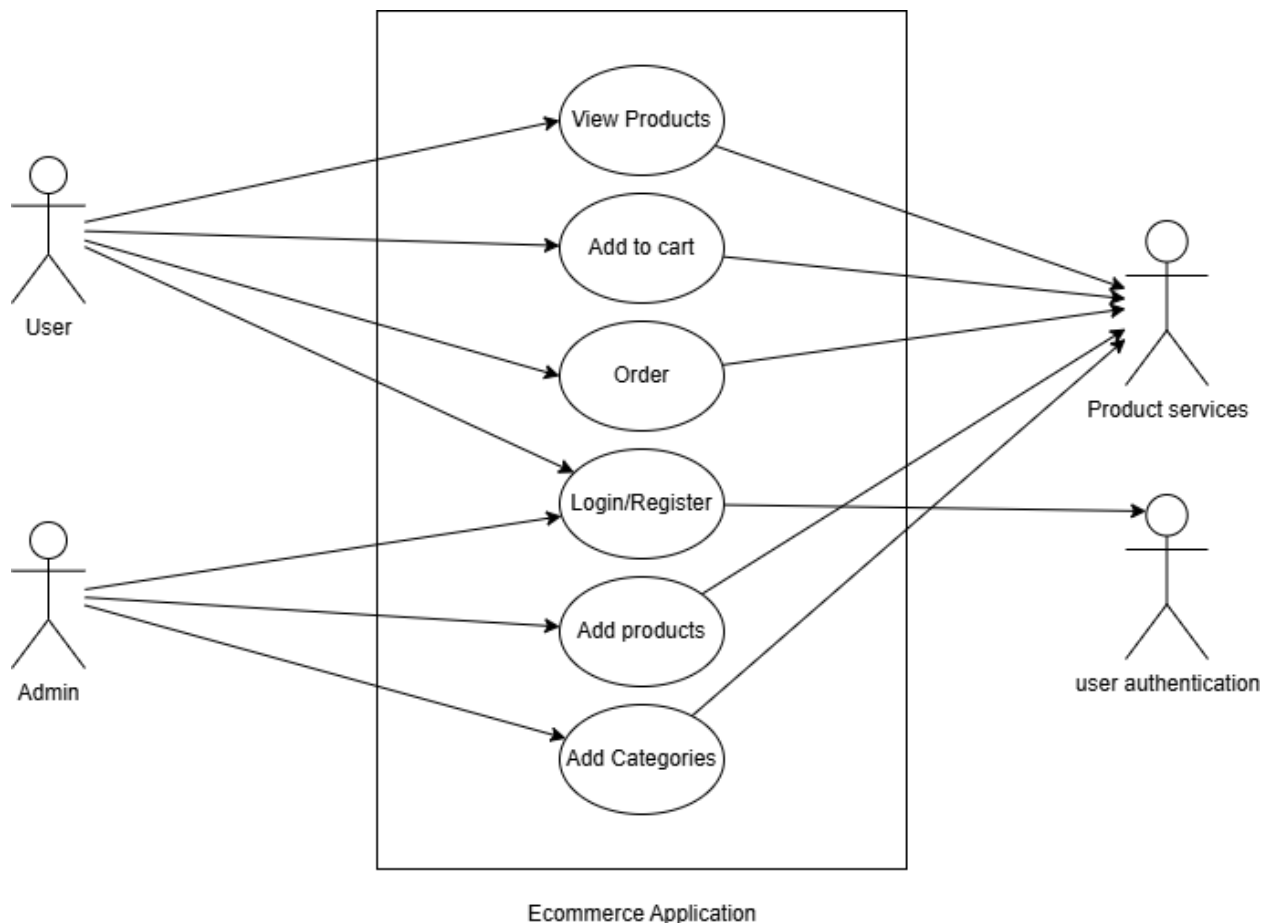


Figure 1.3: Use Case diagram

Feature Set

This eCommerce backend application provides a robust and scalable set of features designed to manage users, products, orders, and authentication. Below is a breakdown of the key feature sets:

User Authentication & Authorization

- User Registration: Allows new users to create an account with email and password.
- User Login with JWT: Authenticates users and generates a secure JWT token for subsequent requests.
- Role-Based Access Control: Distinguishes between regular users and admin users, limiting access based on roles.

Product Management

- Add Product (Admin): Admin can add new products to the system with title, description, price, image URL, category, and rating.
- Update Product (Admin): Admin can modify existing product details.
- Delete Product (Admin): Admin can remove a product from the catalog.
- Get Product List (All Users): Users can view the list of available products.
- Search and Filter Products: Products can be searched by name or filtered by category.

Category Management

- Add Category (Admin): Admin can create product categories (e.g., Electronics, Clothing, etc.).
- View Categories: Categories are used to help users filter and browse products effectively.

Cart Management

- Add to Cart: Users can add one or more products to their personal shopping cart.
- Update Cart Items: Users can change the quantity of items in the cart.
- Remove from Cart: Users can delete products from the cart.
- View Cart: The cart contents are associated with each user and can be retrieved at any time.

Order Management

- Place Order: Users can checkout with the products in their cart.
- View Orders (User): Users can view their own order history, including product details and payment status.
- View Orders (Admin): Admins can view all orders placed across the system for monitoring purposes.

Security Features

- JWT-Based Security: Ensures that only authenticated users can access protected routes.
- Password Encryption: User passwords are encrypted before being stored in the database.
- Access Control: Sensitive operations are restricted based on user roles.

Architecture & Technical Features

- RESTful API Design: Clean, REST-compliant endpoints using standard HTTP verbs (GET, POST, PUT, DELETE).
- Spring Boot Framework: Modular architecture based on Spring Boot, Spring Security, and Spring Data JPA.

- Database Integration: Uses MySQL with Hibernate (JPA) for ORM-based persistence.
- Scalability & Modularity: The application follows layered architecture (Controller, Service, Repository) making it easily extensible.

Table 1.1: Features and their related tables.

Feature Category	Feature Description	Related Table(s)
User Authentication & Authorization	Register/Login, Role-based access via JWT	user, address (from db_users)
Product Management	Add/View/Update/Delete Products	product, productcategory, productmedia, productmediamain, categorymedia, m2m_product_categ
Product Management	View Ratings, Reviews	review, reviewmedia
Category Management	Add/View categories	productcategory, categorymedia, m2m_product_categ
Cart Management	Add to Cart, Update/Delete Items	cart, cartitem
Order Management	Place Order, View Orders	cartorder, cartitem, cart
Security Features	JWT Auth, Password Encryption	Handled via user and application-level config
Architecture/Integration	Microservice DB separation	db_products and db_users

Class Diagrams

The project is structured in a modular format using the standard Spring Boot package hierarchy. The primary base package is ``com.example.eCommerceWebsite``, containing multiple sub-packages to organize code according to functionality.

The major packages include:

- configs – Configuration-related classes such as JWT and database separation
- controller – REST controllers handling HTTP requests and responses.
- dtos – Data Transfer Objects used for request and response modeling.
- Exception – Custom exception handling for service and API layers.
- listeners – Event-driven functionality handlers.
- models – Domain models/entities mapped to database tables.
- repository – Interfaces extending JPARepository for DB operations.
- services – Business logic and application-level services.

Model Layer Breakdown

The models are divided into two sub-packages for better separation of concerns:

- productModel – Contains all entities related to products, media, cart, and reviews. This is stored in **db_products** database
- userModel – Contains entities related to users and their address and role. This is stored in **db_users** database.

Cart Package

- Cart – Represents a shopping cart belonging to a user.
- CartItem – Represents individual items in a cart.
- CartOrder – Represents an order placed from the cart.

Media Package

- CategoryMedia – Media associated with a product category.
- MediaType – Enum for differentiating types of media (image, video, etc.).
- ProductMedia – Media associated with a product (e.g., additional images).
- ProductMediaMain – Primary image or media for product display.
- ReviewMedia – Media added in product reviews.

Product & Category

- Product – Represents the actual product entity.
- ProductCategory – Category to which a product belongs.
- Review – Represents a product review posted by a user.

User Model

- User – Represents a system user (customer/admin).
- Role – Enum or table representing roles assigned to users.
- Address – User's shipping/billing address.

Shared/Utility Models

- **BaseModel** – Abstract base entity used for timestamping and common fields.
- **ErrorResponseBody** – DTO to wrap error responses for REST APIs.
- **MediaBase** – Abstract model inherited by all media types.

User Class Diagram

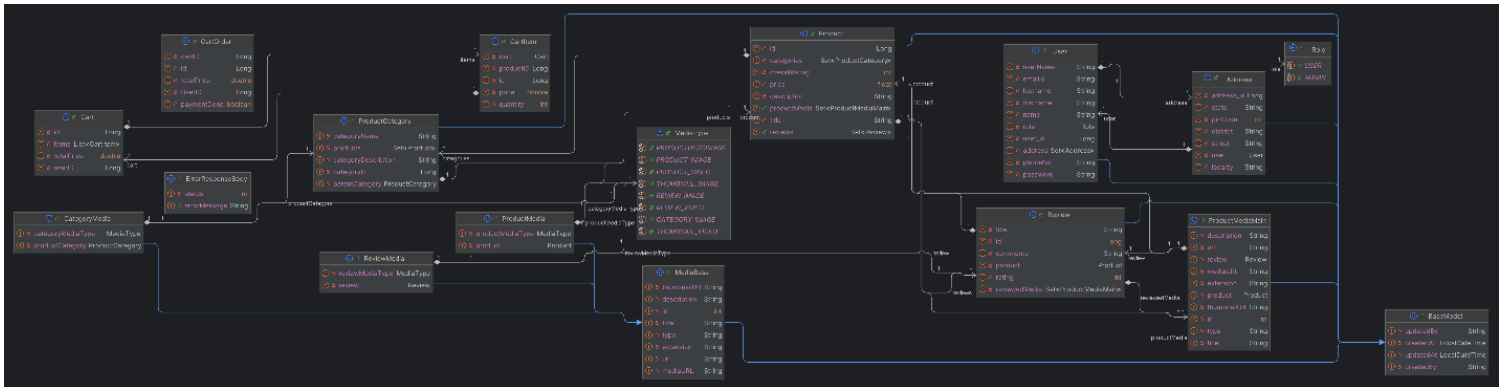


Figure 2.1: User Class Diagram

1. User Class

Attributes:	Relationships:
• id • name • email • password • role • phone • createdAt • updatedAt	• One-to-many with Address (a user can have multiple addresses) • One-to-many with CartOrder (a user can place multiple orders) • One-to-many with Review (a user can write multiple reviews)

2. Address Class

Attributes:	Relationships:
• id • street • city • state • postalCode • country	• Many-to-one with User (belongs to a user)

3. Product Class

Attributes: • id • name • description • price • stock • createdAt • updatedAt	Relationships: • Many-to-many with ProductCategory via m2m_product_category • One-to-many with ProductMediaMain and ProductMedia • One-to-many with Review • One-to-many with CartItem
--	---

4. ProductCategory Class

Attributes: • id • name • description	Relationships: • Many-to-many with Product • One-to-many with CategoryMedia
--	--

5. ProductMedia & ProductMediaMain Classes

Attributes: • id • mediaUrl • type	Relationships: • Many-to-one with Product
---	--

6. CategoryMedia Class

Attributes: • id • mediaUrl • type	Relationships: • Many-to-one with ProductCategory
---	--

7. Review Class

Attributes: • id • rating • comment • createdAt	Relationships: • Many-to-one with User and Product • One-to-many with ReviewMedia
--	--

8. ReviewMedia Class

Attributes:	Relationships:
- id - mediaUrl - type	Many-to-one with Review

9. Cart Class

Attributes:	Relationships:
- id - userId (foreign key)	One-to-many with CartItem

10. CartItem Class

Attributes:	Relationships:
- id - quantity	Many-to-one with Cart and Product

11. CartOrder Class

Attributes:	Relationships:
- id - orderDate - totalAmount - status	- Many-to-one with User - One-to-many with CartItem

12. Role Class

Attributes:	Relationships:
- id - name	One-to-many with User (each user has a role)

Controller Class Diagram

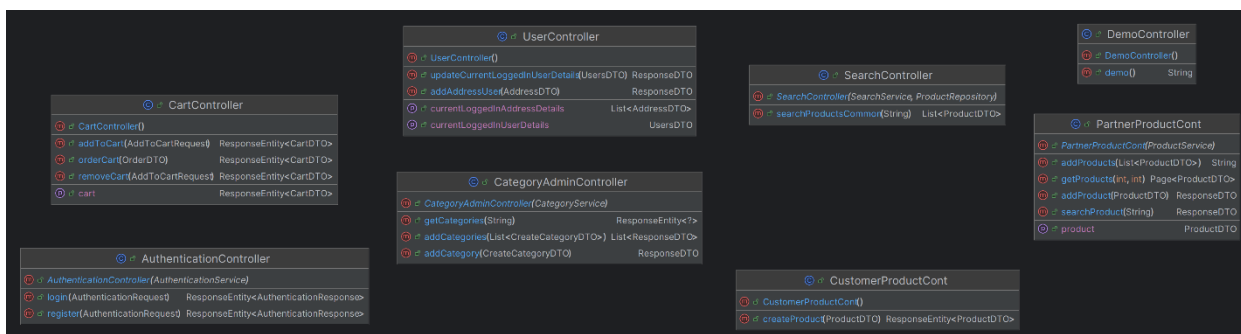


Figure 2.2: Controller Class Diagram

1. AuthenticationController

- **Purpose:** Manages user authentication processes, including user login and registration.
- **Dependencies:**
 - **AuthenticationService:** Injected via the constructor to handle the core authentication logic.
- **Methods:**
 - **login(AuthenticationRequest):** Handles user login attempts. It accepts an AuthenticationRequest object and returns a ResponseEntity containing an AuthenticationResponse.
 - **register(AuthenticationRequest):** Manages new user registration. It takes an AuthenticationRequest and returns a ResponseEntity with an AuthenticationResponse.

2. UserController

- **Purpose:** Handles operations related to managing a user's profile and data.
- **Fields:**
 - **currentUserLoggedInAddressDetails:** A List<AddressDTO> that holds the addresses for the currently logged-in user.
 - **currentUserLoggedInUserDetails:** A UserDTO object containing the profile details of the logged-in user.
- **Methods:**
 - **updateCurrentUserLoggedInUserDetails(UserDetailsDTO):** Updates the profile information of the current user. It accepts a UserDetailsDTO and returns a generic ResponseDTO.
 - **addAddress(UserAddressDTO):** Adds a new shipping or billing address to the user's profile. It accepts a UserAddressDTO and returns a ResponseDTO.

3. CartController

- **Purpose:** Manages all functionalities related to the user's shopping cart.
- **Fields:**
 - **cart:** A ResponseEntity<CartDTO> that likely represents the current state of the shopping cart.
- **Methods:**
 - **addToCart(AddToCartRequest):** Adds a specified item to the shopping cart. It takes an AddToCartRequest and returns a ResponseEntity containing the updated CartDTO.

- **order(CartDTO):** Initiates the checkout process to place an order. It takes a CartDTO and returns a ResponseEntity with the resulting CartDTO.
- **removeCardAddToCartRequest(...):** Removes an item from the cart. (*Note: The method name in the diagram appears incomplete or incorrect.*)

4. CategoryAdminController

- **Purpose:** Provides administrative endpoints for managing product categories.
- **Dependencies:**
 - CategoryService: Injected via the constructor to handle business logic for categories.
- **Methods:**
 - **getCategories(String):** Retrieves a list of product categories, possibly with a filter parameter. It returns a ResponseEntity.
 - **addCategories(List<CreateCategoryDTO>):** Allows for adding multiple new categories in a single request. It returns a List<ResponseDTO>.
 - **addCategory(CreateCategoryDTO):** Adds a single new product category. It returns a ResponseDTO.

5. SearchController

- **Purpose:** Dedicated to handling product search queries from users.
- **Dependencies:**
 - SearchService
 - SearchRepository: Both are injected via the constructor to perform the search operations.
- **Methods:**
 - **searchProductByName(String):** Searches for products based on a provided name string. It returns a List<ProductDTO>.

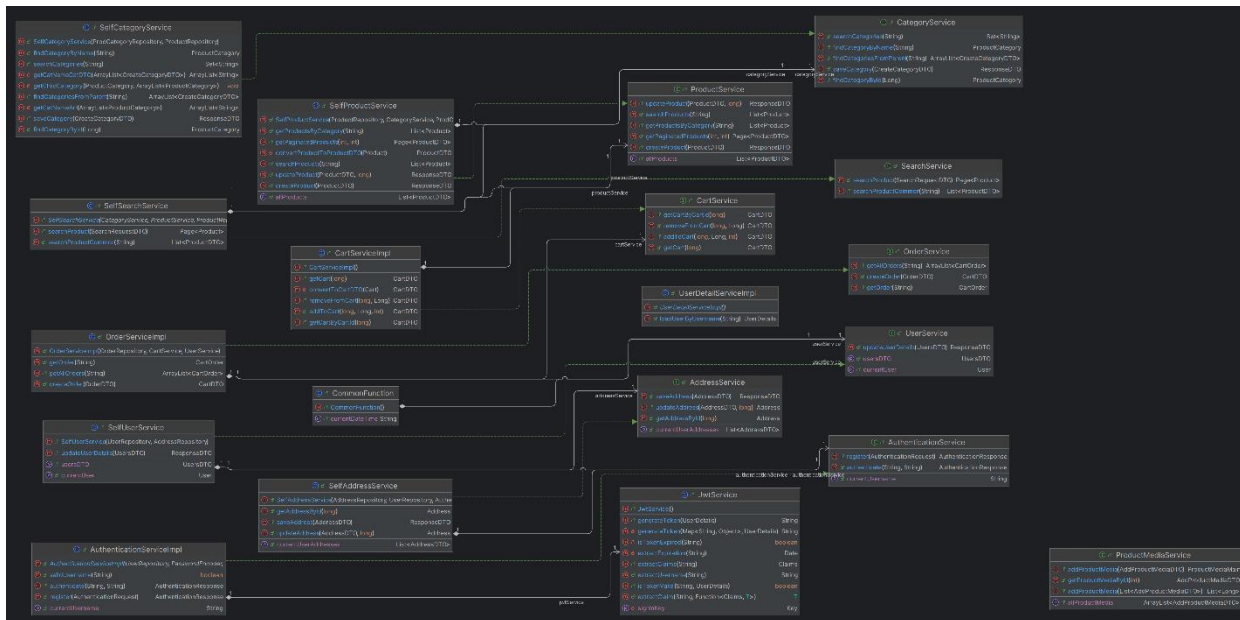
6. PartnerProductController

- **Purpose:** Manages product inventory and details from the perspective of a partner, seller, or vendor.
- **Dependencies:**
 - ProductService: Injected to handle the underlying product logic.
- **Fields:**
 - **product:** A ProductDTO object.
- **Methods:**

- ## 7. CustomerProductController

- ## 8. DemoController

- ## Service Class Diagram



21

SetCategoryService

- Purpose: Responsible for setting/managing product categories.
- Attributes:
 - setCategoryDto: SetCategoryDto (type SetCategoryDto)
- Methods:
 - setCategory(SetCategoryDto setCategoryDto): String
 - setCategories(List<String> categoryDtoList): String
 - deleteCategory(String categoryId): String
 - deleteCategorys(List<String> categoryIdList): String
 - updateCategory(SetCategoryDto setCategoryDto): String
 - updateCategorys(List<SetCategoryDto> setCategoryDtoList): String
 - getCategory(String categoryId): ProductCategory
 - getCategorys(List<String> categoryIdList): List<ProductCategory>

CategoryService

- Purpose: Provides read-only access to product categories.
- Methods:
 - getCategory(String categoryId): ProductCategory
 - getCategoryByName(String categoryName): ProductCategory
 - getCategorys(List<String> categoryIdList): List<ProductCategory>
 - getCategorysByName(List<String> categoryNameList): List<ProductCategory>

SetProductService

- Purpose: Manages the creation, update, and deletion of product information.
- Attributes:
 - setProductDto: SetProductDto (type SetProductDto)
- Methods:
 - setProduct(SetProductDto setProductDto): String
 - setProducts(List<SetProductDto> setProductDtoList): String
 - deleteProduct(String productId): String

- deleteProducts(List<String> productIdList): String
- updateProduct(SetProductDto setProductDto): String
- updateProducts(List<SetProductDto> setProductDtoList): String
- getProduct(String productId): ProductDto
- getProducts(List<String> productIdList): List<ProductDto>

ProductService

- Purpose: Provides read-only access to product details.
- Methods:
 - getProduct(String productId): ProductDto
 - getProducts(List<String> productIdList): List<ProductDto>
 - getProductsByCategory(String categoryId): List<ProductDto>
 - getProductsByBrand(String brandId): List<ProductDto>
 - getProductsByProductType(String productType): List<ProductDto>

SearchService

- Purpose: Handles searching for products.
- Methods:
 - search(String query): List<ProductDto>
 - searchByCategory(String category): List<ProductDto>
 - searchByProduct(String product): List<ProductDto>

CartService

- Purpose: Manages user shopping carts.
- Methods:
 - addToCart(String userId, String productId, Integer quantity): String
 - removeFromCart(String userId, String productId): String
 - updateCart(String userId, String productId, Integer quantity): String
 - getCart(String userId): List<CartItem>

UserService

- Purpose: Provides read-only access to user details.

- Methods:
 - `getUser(String userId): User`
 - `getUserDetails(String userId): UserDetails`

UserDetailsServiceComp

- Purpose: Appears to be a component for managing user details.
- Methods:
 - `getUserDetails(String userId): UserDetails`
 - `setUserDetails(UserDetail userDetails): String`
 - `updateUserDetails(UserDetail userDetails): String`
 - `deleteUserDetails(String userId): String`

OrderService

- Purpose: Manages orders in the system.
- Methods:
 - `createOrder(String userId, List<CartItem> cartItems): String`
 - `getOrder(String orderId): Order`
 - `getOrders(String userId): List<Order>`

OrderServiceComp

- Purpose: A component for order-related operations.
- Methods:
 - `createOrder(Order order): String`
 - `updateOrder(Order order): String`
 - `getOrder(String orderId): Order`
 - `getOrders(String userId): List<Order>`

AuthService

- Purpose: Handles authentication and authorization.
- Methods:
 - `authenticate(String username, String password): String (likely a token)`
 - `authorize(String token, String role): boolean`

SetSearchService

- Purpose: Manages search indexing or configuration.
- Attributes:
 - setSearchDto: SetSearchDto (type SetSearchDto)
- Methods:
 - setSearch(SetSearchDto setSearchDto): String
 - setSearchs(List<SetSearchDto> setSearchDtoList): String
 - updateSearch(SetSearchDto setSearchDto): String
 - deleteSearch(String searchId): String

SetUserService

- Purpose: Manages the creation, update, and deletion of user accounts.
- Attributes:
 - setUserDto: SetUserDto (type SetUserDto)
- Methods:
 - setUser(SetUserDto setUserDto): String
 - setUsers(List<SetUserDto> setUserDtoList): String
 - updateUser(SetUserDto setUserDto): String
 - deleteUser(String userId): String

SetAddressService

- Purpose: Manages user address information.
- Attributes:
 - setAddressDto: SetAddressDto (type SetAddressDto)
- Methods:
 - setAddress(SetAddressDto setAddressDto): String
 - setAddressss(List<SetAddressDto> setAddressDtoList): String
 - updateAddress(SetAddressDto setAddressDto): String
 - deleteAddress(String addressId): String

AddressService

- Purpose: Provides read-only access to address details.
- Methods:
 - getAddress(String addressId): Address
 - getAddresses(String userId): List<Address>

AuthServiceComp

- Purpose: Appears to be a component handling authentication concerns.
- Methods:
 - authenticate(AuthenticationRequest authenticationRequest): AuthenticationResponse
 - authorize(String token): boolean

AuthenticationServiceImp

- Purpose: An implementation of an authentication service.
- Attributes:
 - userName: String
 - password: String
 - token: String
- Methods:
 - authenticate(AuthenticationRequest authenticationRequest): AuthenticationResponse

SetAuthService

- Purpose: Manages authentication-related settings or user roles.
- Methods:
 - setAuth(Auth auth): String
 - updateAuth(Auth auth): String
 - deleteAuth(String authId): String

Auth

- Purpose: Represents authentication data.
- Attributes:
 - authId: String
 - authType: String

- authValue: String
- isEnabled: boolean
- createdAt: Date
- updatedAt: Date

ProductMediaService

- Purpose: Manages product media (images, videos).
- Methods:
 - addProductMedia(String productId, ProductMedia productMedia): String
 - deleteProductMedia(String productMediaId): String
 - getProductMedias(String productId): List<ProductMedia>

CommonFunction

- Purpose: A utility class or service providing common functions.
- Methods:
 - generateUUID(): String
 - getCurrentTimeStamp(): Long

Database Schema Design

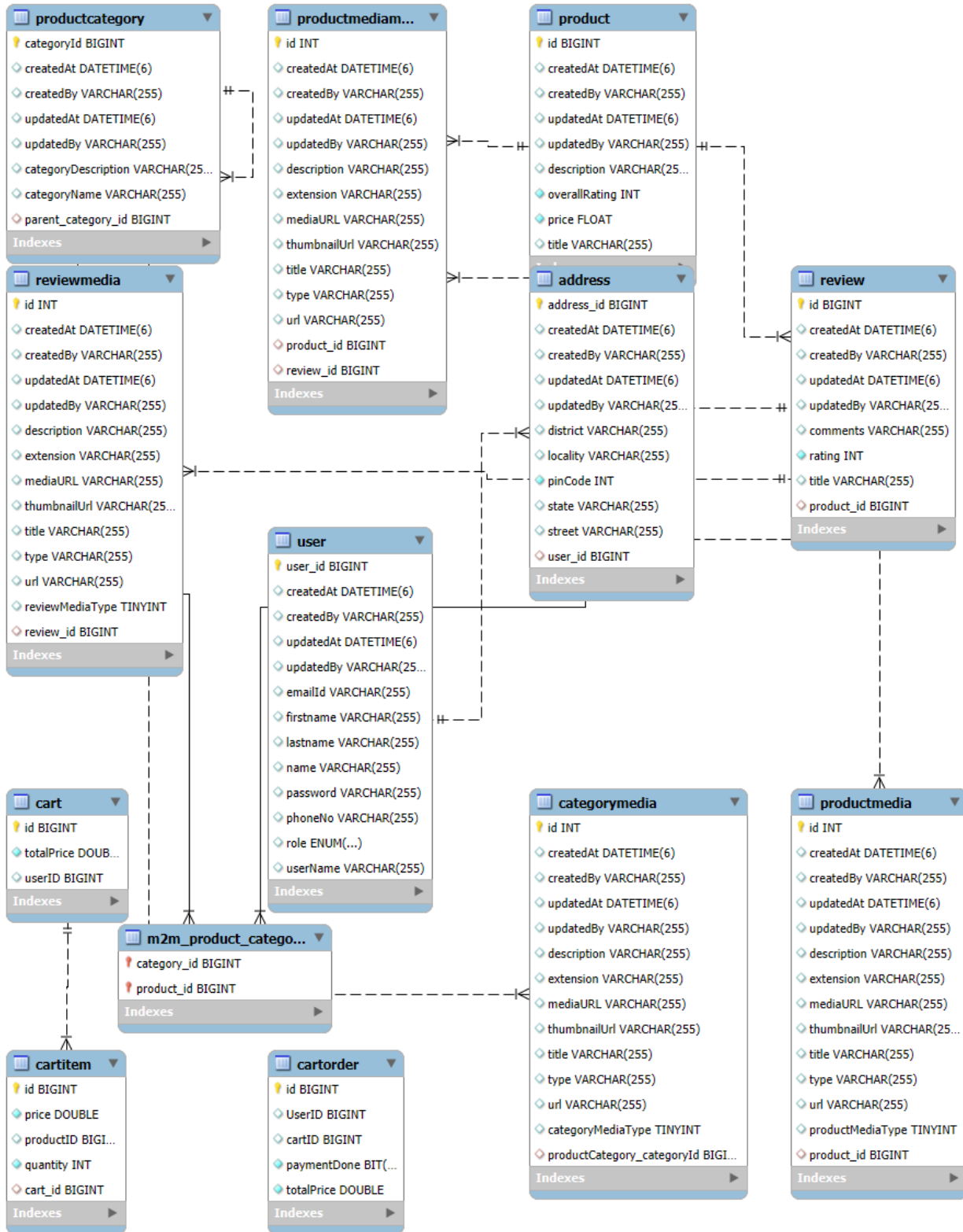


Figure 3.1: Database schema

This database schema is designed to support an e-commerce platform, managing products, categories, users, reviews, carts, and orders.

1. productcategory

- **Purpose:** Stores information about different product categories.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the product category.
 - createdDt (DATETIME(6)): Timestamp when the category was created.
 - createdBy (VARCHAR(255)): User who created the category.
 - updatedDt (DATETIME(6)): Timestamp when the category was last updated.
 - updatedBy (VARCHAR(255)): User who last updated the category.
 - description (VARCHAR(255)): A brief description of the category.
 - categoryName (VARCHAR(255)): Name of the product category (e.g., "Electronics", "Clothing").
 - parent_category_id (BIGINT): Foreign Key referencing productcategory.id. Used to establish hierarchical categories (e.g., "Laptops" under "Electronics").

2. product

- **Purpose:** Stores details about individual products.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the product.
 - createdDt (DATETIME(6)): Timestamp when the product was created.
 - createdBy (VARCHAR(255)): User who created the product.
 - updatedDt (DATETIME(6)): Timestamp when the product was last updated.
 - updatedBy (VARCHAR(255)): User who last updated the product.
 - description (VARCHAR(255)): Detailed description of the product.
 - overallRating (INT): Average rating of the product based on reviews.
 - price (FLOAT): The selling price of the product.

3. user

- **Purpose:** Stores information about registered users of the platform.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the user.
 - createdDt (DATETIME(6)): Timestamp when the user account was created.
 - createdBy (VARCHAR(255)): (Likely system or admin) who created the user.
 - updatedDt (DATETIME(6)): Timestamp when the user account was last updated.
 - updatedBy (VARCHAR(255)): User who last updated the account (or system).
 - emailId (VARCHAR(255)): User's email address (should be unique).
 - firstname (VARCHAR(255)): User's first name.
 - lastname (VARCHAR(255)): User's last name.
 - name (VARCHAR(255)): Full name of the user.
 - password (VARCHAR(255)): Hashed password for the user.
 - phoneNo (VARCHAR(255)): User's phone number.
 - role (ENUM): User's role (e.g., 'CUSTOMER', 'ADMIN').
 - userName (VARCHAR(255)): User's unique username.

4. address

- **Purpose:** Stores address details, typically associated with users for shipping or billing.

- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the address.
 - createdDt (DATETIME(6)): Timestamp when the address was created.
 - createdBy (VARCHAR(255)): User who created the address entry.
 - updatedDt (DATETIME(6)): Timestamp when the address was last updated.
 - updatedBy (VARCHAR(255)): User who last updated the address entry.
 - description (VARCHAR(255)): Additional description for the address.
 - district (VARCHAR(255)): District of the address.
 - locality (VARCHAR(255)): Locality or area of the address.
 - pinCode (INT): Pin code or postal code of the address.
 - state (VARCHAR(255)): State of the address.
 - street (VARCHAR(255)): Street name/number of the address.
 - user_id (BIGINT): Foreign Key referencing user.id. Links an address to a specific user.

5. review

- **Purpose:** Stores customer reviews for products.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the review.
 - createdDt (DATETIME(6)): Timestamp when the review was created.
 - createdBy (VARCHAR(255)): User who created the review.
 - updatedDt (DATETIME(6)): Timestamp when the review was last updated.
 - updatedBy (VARCHAR(255)): User who last updated the review.
 - comments (VARCHAR(255)): The actual text of the review.
 - rating (INT): The numerical rating given by the user (e.g., 1 to 5 stars).
 - title (VARCHAR(255)): Title of the review.
 - product_id (BIGINT): Foreign Key referencing product.id. Links the review to a specific product.

6. productmedia

- **Purpose:** Stores metadata about media (images, videos) associated with products.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the product media.
 - createdDt (DATETIME(6)): Timestamp when the media entry was created.
 - createdBy (VARCHAR(255)): User who created the media entry.
 - updatedDt (DATETIME(6)): Timestamp when the media entry was last updated.
 - updatedBy (VARCHAR(255)): User who last updated the media entry.
 - extension (VARCHAR(255)): File extension of the media (e.g., "jpg", "mp4").
 - mediaURL (VARCHAR(255)): URL to the full-size media file.
 - thumbnailURL (VARCHAR(255)): URL to a thumbnail version of the media.
 - title (VARCHAR(255)): Title or caption for the media.
 - type (VARCHAR(255)): Type of media (e.g., "image", "video").
 - url (VARCHAR(255)): General URL for the media (could be same as mediaURL).
 - productMediaType (TINYINT): (Specific media type code).
 - product_id (BIGINT): Foreign Key referencing product.id. Links the media to a specific product.

7. reviewmedia

- **Purpose:** Stores metadata about media associated with reviews.
- **Columns:**

- id (BIGINT): Primary Key. Unique identifier for the review media.
- createdDt (DATETIME(6)): Timestamp when the media entry was created.
- createdBy (VARCHAR(255)): User who created the media entry.
- updatedDt (DATETIME(6)): Timestamp when the media entry was last updated.
- updatedBy (VARCHAR(255)): User who last updated the media entry.
- description (VARCHAR(255)): Description of the review media.
- extension (VARCHAR(255)): File extension of the media.
- mediaURL (VARCHAR(255)): URL to the full-size media file.
- thumbnailURL (VARCHAR(255)): URL to a thumbnail version.
- title (VARCHAR(255)): Title for the media.
- type (VARCHAR(255)): Type of media.
- url (VARCHAR(255)): General URL.
- reviewMediaType (TINYINT): (Specific media type code).
- review_id (BIGINT): Foreign Key referencing review.id. Links media to a specific review.

8. categorymedia

- **Purpose:** Stores metadata about media associated with product categories.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the category media.
 - createdDt (DATETIME(6)): Timestamp when the media entry was created.
 - createdBy (VARCHAR(255)): User who created the media entry.
 - updatedDt (DATETIME(6)): Timestamp when the media entry was last updated.
 - updatedBy (VARCHAR(255)): User who last updated the media entry.
 - extension (VARCHAR(255)): File extension of the media.
 - mediaURL (VARCHAR(255)): URL to the full-size media file.
 - thumbnailURL (VARCHAR(255)): URL to a thumbnail version.
 - title (VARCHAR(255)): Title for the media.
 - type (VARCHAR(255)): Type of media.
 - url (VARCHAR(255)): General URL.
 - categoryMediaType (TINYINT): (Specific media type code).
 - product_category_id (BIGINT): Foreign Key referencing productcategory.id. Links media to a specific product category.

9. cart

- **Purpose:** Represents a shopping cart for a user.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the cart.
 - totalPrice (DOUBLE): The total price of all items in the cart.
 - userId (BIGINT): Foreign Key referencing user.id. Links the cart to a specific user.

10. cartitem

- **Purpose:** Stores individual items within a shopping cart.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the cart item.
 - product_id (BIGINT): Foreign Key referencing product.id. Links the cart item to a specific product.
 - quantity (INT): The quantity of the product in the cart.
 - cart_id (BIGINT): Foreign Key referencing cart.id. Links the cart item to a specific cart.

11. cartorder

- **Purpose:** Stores information about completed orders, likely derived from a cart.
- **Columns:**
 - id (BIGINT): Primary Key. Unique identifier for the order.
 - userId (BIGINT): Foreign Key referencing user.id. Links the order to the user who placed it.
 - paymentDone (BOOLEAN): Indicates if the payment for the order has been completed.
 - totalPrice (DOUBLE): The total price of the order.

12. m2m_product_category (Many-to-Many Join Table)

- **Purpose:** Resolves the many-to-many relationship between product and productcategory, allowing a product to belong to multiple categories and a category to contain multiple products.
- **Columns:**
 - category_id (BIGINT): Foreign Key referencing productcategory.id.
 - product_id (BIGINT): Foreign Key referencing product.id.
- **Primary Key:** Composite Primary Key (category_id, product_id).

Feature Development Process

Search Products

The "Search Products" feature is fundamental for any e-commerce platform, enabling users to quickly locate desired items from a vast catalog.

The core user need is to efficiently find products. A basic search would allow users to type keywords and get relevant product listings. Initially, this might involve simple string matching against product names and category. As the platform scales, more advanced capabilities like relevance ranking, filtering, and typo tolerance become essential.

Backend Request Flow: From Frontend to Database

Let's trace a typical search request from a user's browser to the backend services.

API Request Payload

When a user types "laptop" into the search bar and hits enter, the frontend (e.g., a React application) constructs an API request.

- HTTP Method: GET (for retrieving data)
- Endpoint: /search
- Query Parameters (for GET request):
- GET /search?query=phone

Service Which Picks the Request

Upon receiving the API request, the backend web server (e.g., Node.js with Express, Spring Boot, Django) routes it to the appropriate controller method.

Flow of MVC Architecture

1. Controller :
 - Receives the query from the Rest API Controller.
 - Process it and then sends it to the Service
2. Service :
 - Service validates the query string and searches the product database and will find out if the query string is a category or a product.
 - If it's a category, it will find all the subcategories under it and will return all the products from these categories
 - If it's a product, will search the product table directly and will return the products that matches the query keyword.

3. Repository:

- The service uses product repository and category repository to search for products from the database

Deployment Flow

Ecommerce web application file is built using **Maven**, which compiles the Java code and packages it into an executable **JAR (Java Archive)** file. This JAR file contains all the necessary dependencies to run the application independently.

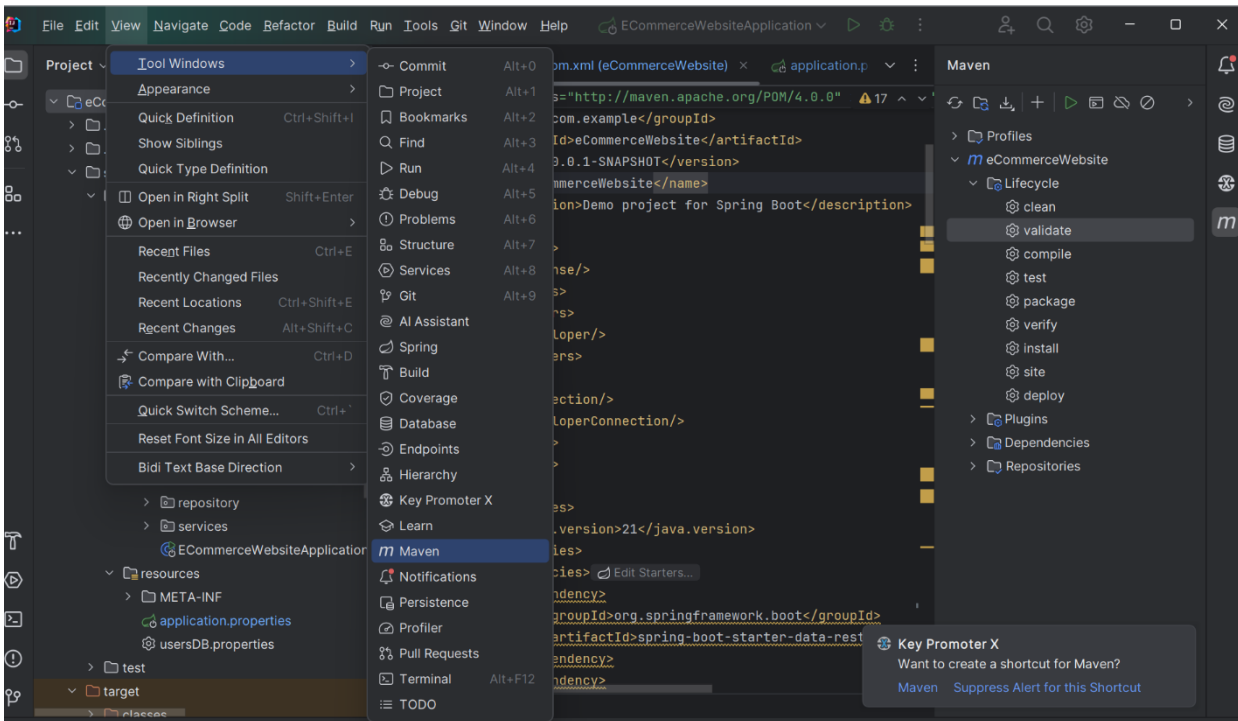


Figure 4.1: converting a project to JAR

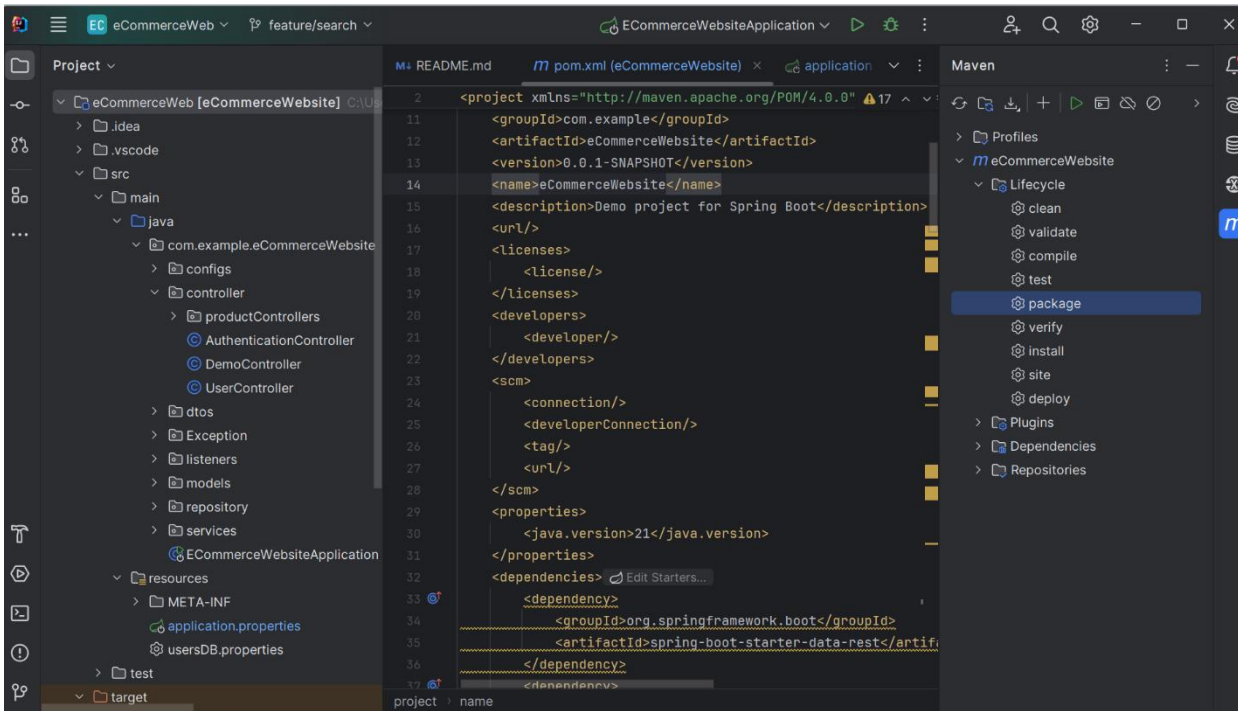


Figure 4.2: Converted to JAR

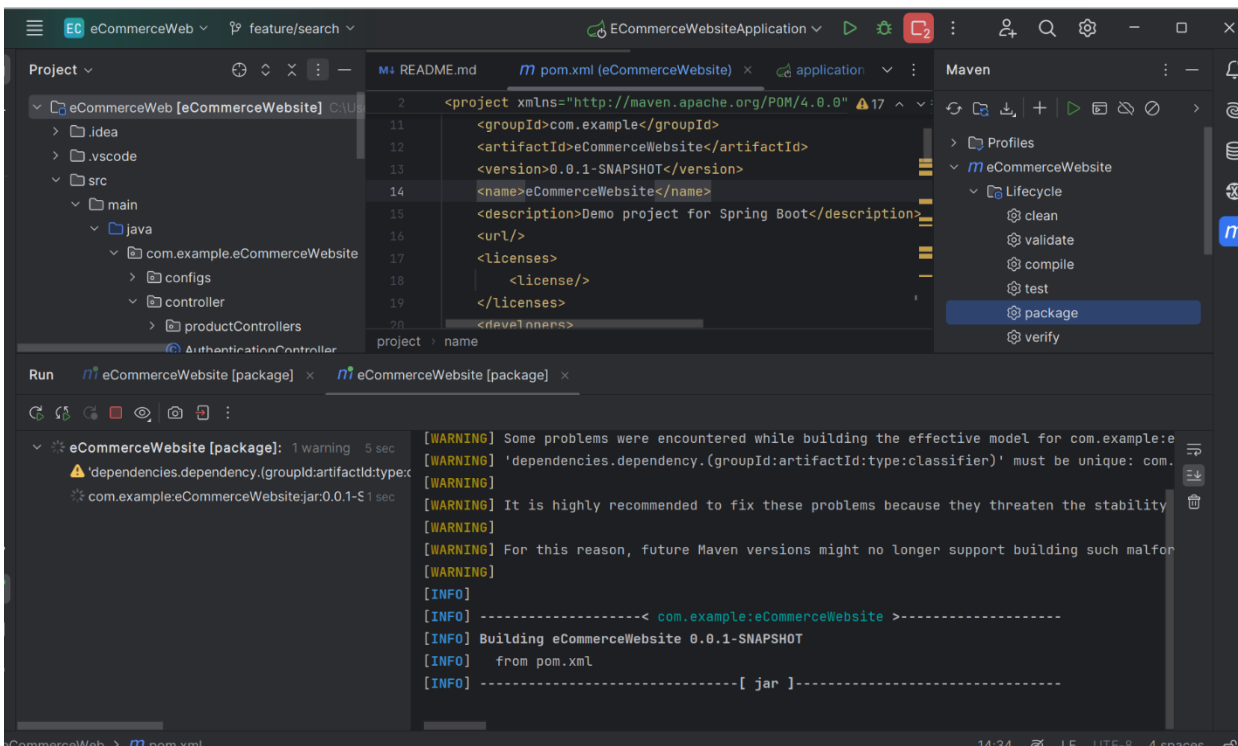


Figure 4.3: JAR File

Elastic Beanstalk: Application Hosting and Management

AWS Elastic Beanstalk serves as the Platform as a Service (PaaS) for deploying and managing the Spring Boot application. The deployment process to Elastic Beanstalk involves the following key steps:

1. **Application Upload:** The generated Ecommerce app JAR file is uploaded to Elastic Beanstalk via the AWS Management Console, AWS Command Line Interface (CLI), or an IDE plugin. This file is stored in an S3 bucket managed internally by Elastic Beanstalk.
2. **Environment Provisioning:** Upon receiving the application package, Elastic Beanstalk automatically provisions the necessary AWS resources to run the Spring Boot application. These resources typically include:
 - **Amazon EC2 Instances:** Virtual servers where the Spring Boot application runs. Elastic Beanstalk handles the instance selection, operating system (e.g., Amazon Linux 2 with Corretto Java), and initial configuration.
 - **Application Load Balancer (ALB):** (If configured for high availability or scalability) An ALB is provisioned to distribute incoming web traffic across multiple EC2 instances, ensuring even load distribution and fault tolerance.
 - **Auto Scaling Group:** An Auto Scaling group is created to dynamically adjust the number of EC2 instances based on predefined metrics (e.g., CPU utilization). This ensures the application can scale out during peak demand and scale in during low usage, optimizing costs.
 - **Security Groups:** Network security rules are configured to control inbound and outbound traffic for both the EC2 instances and the Load Balancer, enhancing the security posture of the application.
 - **CloudWatch Integration:** Elastic Beanstalk integrates with Amazon CloudWatch for monitoring the health and performance of the application and its underlying resources, allowing for the creation of alarms and metrics.
3. **Application Deployment and Health Checks:** Elastic Beanstalk deploys the Spring Boot application onto the provisioned EC2 instances, ensuring it starts correctly and is accessible. Continuous **health checks** are performed to monitor the application's responsiveness. If an instance becomes unhealthy, Elastic Beanstalk can automatically replace it.

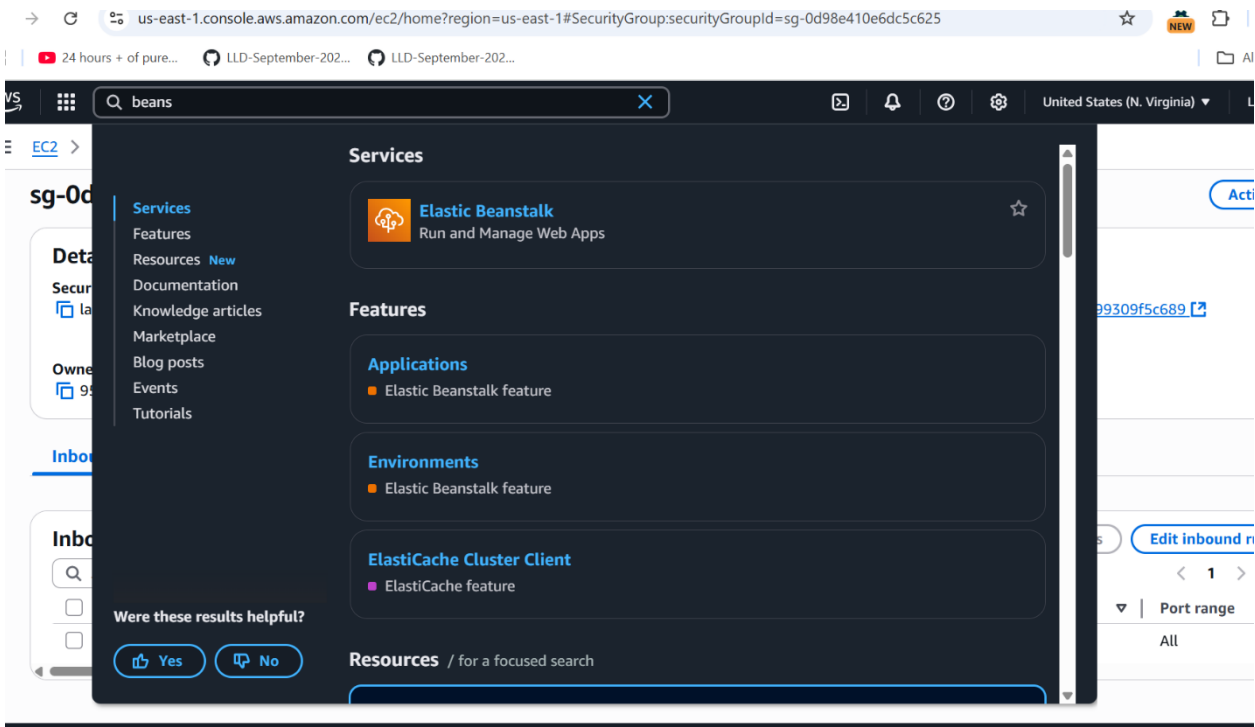


Figure 5.1: AWS Beanstalk

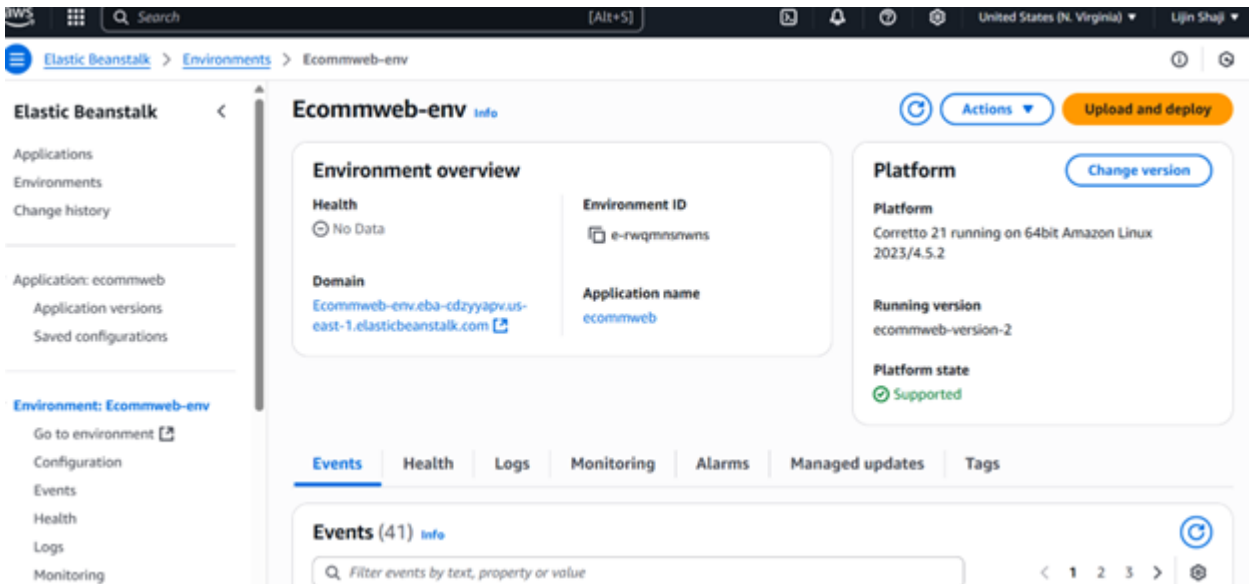


Figure 5.2: Beanstalk environment before deploy

Amazon RDS for MySQL: Managed Database Service

The e-commerce application utilizes **Amazon Relational Database Service (RDS)** to host its MySQL database. RDS is a managed service that simplifies the setup, operation, and scaling of relational databases, abstracting away the complexities of database administration.

1. **Database Instance Provisioning:** A MySQL database instance is provisioned within RDS. AWS manages routine tasks such as patching, backups, and replication, ensuring database reliability and availability.

2. **Secure Connection:** The Spring Boot application, running on the EC2 instances managed by Elastic Beanstalk, establishes a secure connection to the RDS MySQL instance. Database connection details (host, port, username, password) are typically configured within the Spring Boot application.properties or application.yml file, often referencing **environment variables** managed by Elastic Beanstalk for security.
3. **Network Isolation:** The RDS instance is strategically placed in a **private subnet** within the VPC, restricting direct public access. Access is granted only to the EC2 instances within the Elastic Beanstalk environment via meticulously configured **security group rules** on the MySQL port (3306).

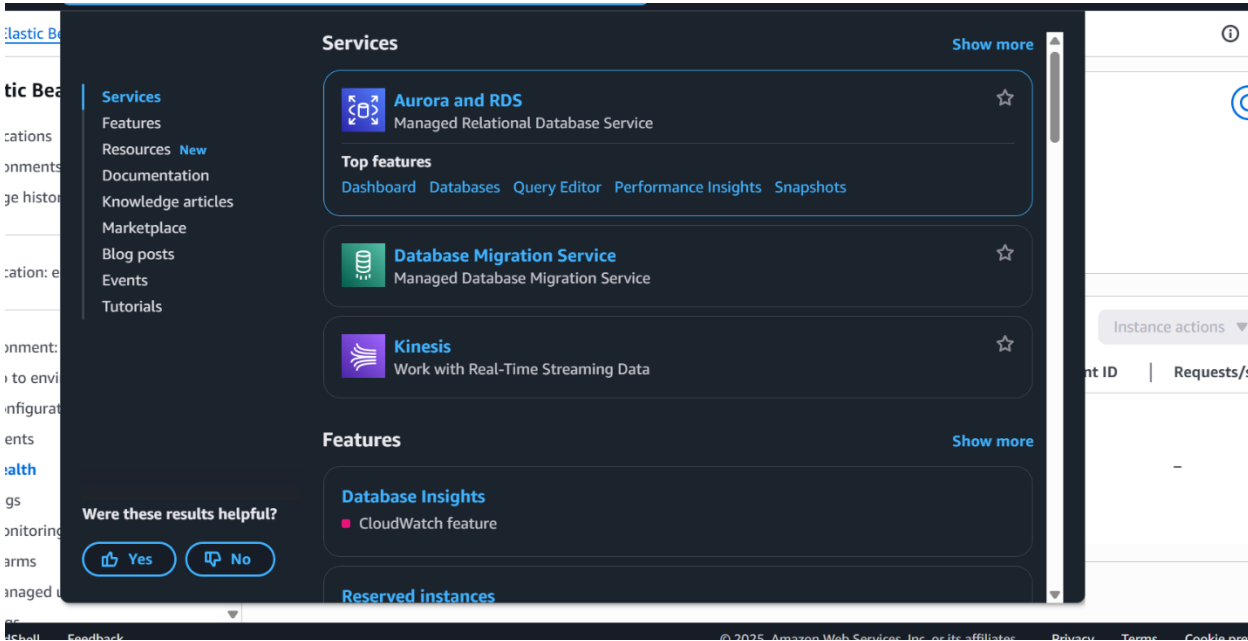
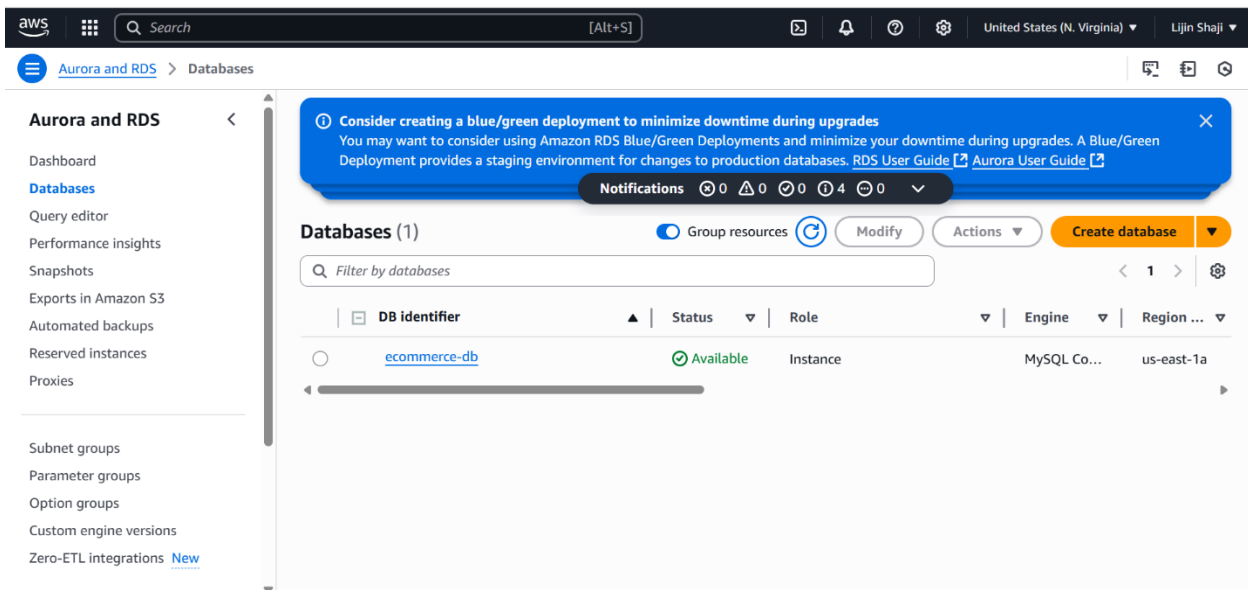


Figure 6.1: MY SQL Creation in AWS RDS



Network and Security Considerations (AWS VPC)

All AWS resources for this deployment operate within an **Amazon Virtual Private Cloud (VPC)**. The VPC acts as a virtual network dedicated to this AWS account, providing isolation and control over the network environment.

VPC Structure

The VPC is logically segmented into **subnets**:

- **Public Subnet:** This subnet hosts resources that require direct internet access, such as the **Application Load Balancer**. It is connected to an **Internet Gateway (IGW)**, which enables communication with the public internet.
- **Private Subnets:** These subnets host resources that should not be directly accessible from the internet, including the **EC2 instances** running the Spring Boot application and the **RDS MySQL instance**. Communication to the internet from private subnets (e.g., for software updates or external API calls) is facilitated via a **NAT Gateway (Network Address Translation Gateway)**.

Security Groups

Security Groups act as virtual firewalls at the instance level, controlling inbound and outbound traffic. Key security group configurations include:

- **Load Balancer Security Group:** Allows inbound traffic from the internet on standard HTTP (port 80) and HTTPS (port 443) ports.
- **EC2 Instance Security Group:** Allows inbound traffic only from the Load Balancer's security group on the application's port (e.g., 8080 for Spring Boot). This prevents direct public access to the application instances.
- **RDS Security Group:** Allows inbound traffic on the MySQL port (3306) only from the EC2 instance's security group, ensuring that only the application can connect to the database.

Accessing the Application

Users access the e-commerce application through the public DNS name provided by the Elastic Beanstalk environment, which can also be mapped to a **custom domain name** for a professional appearance.

The traffic flow is as follows:

1. User's request hits the **Application Load Balancer**.
2. The Load Balancer forwards the request to a healthy **EC2 instance** running the Spring Boot application in a private subnet.

3. The Spring Boot application processes the request, interacts with the **RDS MySQL database** as required (e.g., retrieving product information, processing orders).
4. The application returns the response to the Load Balancer, which then sends it back to the user.

Technologies Used

Programming language: JAVA

- **Description:** A high-level, object-oriented language known for its "**write once, run anywhere**" capability, robustness, and platform independence.
- **Real-life Utility:** Ideal for building **enterprise-scale applications**, **Android mobile apps**, and **big data solutions** due to its mature ecosystem and performance.
- **Real-life Applications:** Core banking systems (**JP Morgan Chase**), Android applications (**Google Maps**, **Spotify**), and big data frameworks (**Apache Hadoop**).

Web Framework: Spring Boot

- **Description:** An opinionated framework that simplifies the creation of **production-ready, stand-alone Spring applications** by reducing configuration overhead with auto-configuration and embedded servers.
- **Real-life Utility:** Accelerates the development of **microservices** and **RESTful APIs**, allowing developers to focus on business logic and enabling **rapid deployment and iteration**.
- **Real-life Applications:** Powering microservices architectures (**Netflix**, **Uber**) and backends for various **SaaS applications** and **e-commerce platforms**.

Persistence Technologies: JPA, Hibernate, and Spring Data JPA

- **Description:**
 - **JPA (Java Persistence API):** A Java API **specification for Object-Relational Mapping (ORM)**, enabling mapping of Java objects to database tables.
 - **Hibernate:** A widely used **ORM framework and an implementation of JPA**, translating Java object interactions into database queries.
 - **Spring Data JPA:** Part of the Spring Data project, it further **simplifies data access layer implementation** by generating repository methods automatically, reducing boilerplate code.
- **Real-life Utility:** Together, they streamline database interactions by allowing developers to work with **objects instead of raw SQL**, improving productivity, code readability, and maintainability.
- **Real-life Applications:** Fundamental to **enterprise Java applications** (e.g., CRM, ERP, financial systems) and backends for **web services and APIs** requiring robust data persistence.

Core Spring Modules: Spring Web (Spring MVC) and Spring Security

- **Description:**
 - **Spring Web / Spring MVC:** Provides the **Model-View-Controller (MVC) pattern** for building web applications and **RESTful APIs**, handling HTTP requests and responses.

- **Spring Security:** A powerful framework for **authentication and authorization**, offering robust security features for Spring applications, including protection against common attacks.
- **Real-life Utility:**
 - **Spring Web:** Enables **rapid development of scalable web services** that serve front-end applications.
 - **Spring Security:** Essential for building **secure applications**, protecting sensitive data, and ensuring only authorized users access functionalities, critical for e-commerce.
- **Real-life Applications:**
 - Spring Web: Utilized by nearly all Spring Boot microservices and API-driven applications.
 - Spring Security: Key for securing enterprise applications, financial platforms, and e-commerce sites for user authentication and access control.

Relational Database: MySQL

- Description: An open-source Relational Database Management System (RDBMS) known for its speed, reliability, and use of SQL for data management.
- Real-life Utility: Ideal for storing structured data and managing relationships in transactional applications where data consistency is crucial.
- Real-life Applications: Powering numerous web applications (WordPress, Drupal), early social media platforms (Facebook), and many e-commerce backends.

Cloud Platform Service: AWS Elastic Beanstalk

- Description: A managed service for deploying and scaling web applications, automating infrastructure provisioning (EC2, load balancing, auto-scaling) based on uploaded code.
- Real-life Utility: Simplifies cloud deployment for developers by abstracting infrastructure management, enabling quick iterations and consistent environments.
- Real-life Applications: Used by startups for MVPs, hosting web applications and APIs, and setting up development/staging environments.

Cloud Database Service: AWS RDS for MySQL

- Description: A managed relational database service that automates setup, operation, and scaling of MySQL instances in the cloud, handling tasks like patching and backups.
- Real-life Utility: Reduces operational overhead for database administration, provides high availability (Multi-AZ), and supports scalable database capacity.
- Real-life Applications: Widely adopted by SaaS applications, enterprises migrating on-premises databases to the cloud, and backends for mobile and web applications.

Build Automation Tool: Apache Maven (or Gradle)

- Description: Tools (like Maven or Gradle) that automate the build process for Java projects, managing dependencies, compiling code, running tests, and packaging applications.
- Real-life Utility: Ensures consistent builds across different environments, simplifies dependency management, and provides a standardized way to package applications for deployment.
- Real-life Applications: Essential for almost all professional Java projects and a fundamental component of Continuous Integration/Continuous Delivery (CI/CD) pipelines.
-

Version Control System: Git

- **Description:** A distributed version control system used for tracking changes in source code, enabling collaborative development and maintaining a complete history of all modifications.
- **Real-life Utility:** Indispensable for team collaboration, managing code changes, merging different development branches, and facilitating code reviews.
- **Real-life Applications:** The de facto standard for version control in virtually all software development teams, integrated into platforms like GitHub, GitLab, and Bitbucket.

Conclusion

This backend e-commerce application, built with **Java Spring Boot**, successfully delivers a robust, secure, and modular solution for managing core retail operations. By meticulously handling product, cart, and order management, alongside implementing **JWT-based authentication** for secure user interactions, the project establishes a strong foundation for a comprehensive online shopping experience.

Leveraging powerful technologies such as **Spring Boot, Spring Security, JPA/Hibernate, and MySQL**, the application adheres to **RESTful principles** and **clean architectural practices**. This approach ensures not only high performance and data integrity but also promotes a **modular, scalable, and easily extendable** system. The choice of these industry-standard technologies means the backend can readily integrate with diverse frontend frameworks or mobile applications, making it a versatile and future-proof solution for modern digital commerce.

In essence, this project demonstrates a complete, secure, and efficient backend infrastructure, ready to support the dynamic needs of an e-commerce platform and evolve with future business requirements.

References

- ✚ **Spring Team.** (n.d.). *Building a RESTful Web Service*. Spring Guides. Accessed June 10, 2025, from <https://spring.io/guides/gs/rest-service/>
- ✚ **Paraschiu, Eugen.** (n.d.). *Spring Boot Security JWT Authentication*. Baeldung. Accessed May 28, 2025, from <https://www.baeldung.com/spring-boot-security-jwt>
- ✚ **Amazon Web Services.** (n.d.). *Deploying a Spring Boot Application to Elastic Beanstalk*. AWS Documentation. Accessed June 05, 2025, from <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/java-se-platform.html>
- ✚ **Oracle Corporation.** (n.d.). *MySQL 8.0 Reference Manual*. MySQL Official Documentation. Accessed April 15, 2025, from <https://dev.mysql.com/doc/refman/8.0/en/>
- ✚ **Spring Team.** (n.d.). *Spring Boot Documentation*. Spring Documentation. Accessed June 11, 2025, from <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- ✚ Gemini, Chat GPT